招生筛选系统 → 商业模式工坊 迁移报告

tsinghua-sem-ai-screener 整合进 business-model-workshop 的完整记录

日期: 2026-03-05　迁移方式: 子模块整合 + 后端重构　状态: 全部完成

📋 一、迁移概览

将独立运行的招生筛选项目（tsinghua-sem-ai-screener）以 **screener/** 子目录的形式整合进商业模式工坊。同时将原来的 Supabase 后端替换为项目自带的 Express + SQLite 后端。

> ✅ **迁移结果：**导航栏右侧新增「招生」入口，点击可进入完整的招生筛选系统（欢迎页 → BID 介绍 → 表单 → 面试 → 结果），管理员登录后可进入后台。12 个配置项全部实现后端持久化。

📁 二、迁移后文件结构

图例： NEW 新增　MOD 修改　原样复制

```
business-model-workshop-20260302-133400/
├── App.tsx                    ← 新增 ScreenerApp 导航入口 + 视图渲染
├── types.ts                   ← AppView enum 新增 Screener
├── index.html                 ← 合并 Tailwind 配置、字体、importmap
├── package.json               ← 新增依赖 @google/genai, recharts
├── .env.local                 ← VITE_BACKEND_URL=http://localhost:300
│
├── backend/
│   ├── db.js                  ← 新增 4 张 screener 数据表 + CRUD 对象
│   ├── server.js              ← 挂载 screenerRoutes
│   └── screenerRoutes.js      ← 新建：全部 screener API 路由
│
```

```
├── screener/                        ← 新建子目录（招生筛选系统）
│   ├── ScreenerApp.tsx              ← 原 App.tsx，重命名 + 改 import
│   ├── types.ts                     ← 原样
│   ├── i18n.ts                      ← 原样
│   │
│   ├── components/                  ← 20 个组件（原样复制）
│   │   ├── AdminAIAssistant.tsx
│   │   ├── AdminCriteria.tsx
│   │   ├── AdminLibrary.tsx
│   │   ├── AdminLogin.tsx
│   │   ├── AdminPrompts.tsx
│   │   ├── AdminQuestions.tsx
│   │   ├── AdminQuickPreview.tsx
│   │   ├── ApiSettings.tsx
│   │   ├── BackupManager.tsx
│   │   ├── BasicInfoForm.tsx
│   │   ├── BatchCalibrationPanel.tsx
│   │   ├── BidIntro.tsx
│   │   ├── ChatInterface.tsx
│   │   ├── DecisionTreeViz.tsx
│   │   ├── ExportColumnSelector.tsx
│   │   ├── HelpWidget.tsx
│   │   ├── ImportInfoPanel.tsx
│   │   ├── OpenEndedAnalysis.tsx
│   │   ├── ResultView.tsx
│   │   └── WelcomeScreen.tsx
│   │
│   ├── services/                    ← 14 个服务（4 个重写，其余原样）
│   │   ├── supabaseClient.ts        ← 重写为通用 apiFetch() 客户端
│   │   ├── settingsService.ts       ← 重写：fetchSetting / saveSetting
│   │   ├── candidateService.ts      ← 重写：CRUD 走 Express API
│   │   ├── backupService.ts         ← 重写：快照同步 Express API
│   │   ├── feedbackService.ts       ← 重写：管理员备注同步后端
│   │   ├── promptVersionService.ts← 重写：提示词版本同步后端
│   │   ├── authService.ts           ← 重写：本地认证（无 Supabase）
│   │   ├── aiService.ts             ← 原样
│   │   ├── llmService.ts            ← 原样
│   │   ├── adaptiveQuestionEngine.ts
│   │   ├── candidateProfiler.ts
│   │   ├── interviewPipeline.ts
│   │   ├── promptAssembler.ts
```

```
│   │   └── scoringService.ts
│   │
│   └── data/                      ← 2 个数据文件（原样复制）
│       ├── exampleCandidates.ts
│       └── exampleQuestions.ts
│
├── QUICK_DEPLOY.sh                ← 一键部署脚本
├── upload-to-server.sh            ← rsync 上传脚本
└── ...
```

🖥️ 三、后端架构（Supabase → Express + SQLite）

3.1 新增 SQLite 数据表（backend/db.js）

表名	用途	主要字段
`screener_help_config`	帮助信息（营业时间、联系方式、备注）	id (固定'default'), hours, contact, notes
`screener_snapshots`	版本备份	id, label, timestamp, data (JSON)
`screener_settings`	通用键值存储（API 配置、权重、阈值等）	id, value (JSON), updated_at
`screener_candidates`	人才库	id, name, email, data (JSON), created_at, updated_at

3.2 API 路由（backend/screenerRoutes.js）

所有路由挂载在 `/api/screener` 前缀下：

GET	/help-config	获取帮助信息
PUT	/help-config	更新帮助信息
GET	/snapshots	获取所有快照
POST	/snapshots	创建快照
DELETE	/snapshots/:id	删除快照
GET	/settings/:id	获取单项配置

PUT	/settings/:id	保存/更新配置
POST	/settings/batch	批量获取配置
GET	/candidates	获取所有候选人
POST	/candidates	新增候选人
PUT	/candidates/:id	更新候选人
DELETE	/candidates/:id	删除候选人
POST	/candidates/bulk	批量导入候选人

🔄 四、数据同步全景（12 项全部后端持久化）

所有 localStorage 配置均实现双写：本地即时生效 + 异步同步后端。应用启动时从后端拉取并与本地合并。

#	localStorage Key	后端存储	同步方式
1	`tsinghua_api_config`	settings/api_config	settingsService
2	`tsinghua_questions`	settings/questions	settingsService
3	`tsinghua_dimension_weights`	settings/dimension_weights	settingsService
4	`tsinghua_decision_thresholds`	settings/decision_thresholds	settingsService
5	`tsinghua_prompt_config`	settings/prompt_config	settingsService
6	`tsinghua_decision_tree`	settings/decision_tree	settingsService
7	`tsinghua_probing_strategy`	settings/probing_strategy	settingsService
8	`tsinghua_workflow_modules`	settings/workflow_modules	settingsService
9	`tsinghua_candidates`	screener_candidates 表	candidateService
10	`tsinghua_snapshots`	screener_snapshots 表	backupService
11	`tsinghua_feedback`	settings/feedback	settingsService

12	`tsinghua_prompt_versions`	settings/prompt_versions	settingsService

🎨 五、前端整合要点

5.1 导航入口（App.tsx）

- `AppView` 枚举新增 `Screener = 'SCREENER'`
- 导航栏右侧分隔线前新增「招生」按钮（图标：`ClipboardCheck`）
- 视图渲染区新增 `{view === AppView.Screener && <ScreenerApp />}`

5.2 滚动修复

`<main>` 容器的 overflow 改为条件式：招生视图用 `overflow-y-auto`（允许滚动），其他视图保持 `overflow-hidden`。

5.3 Tailwind 扩展（index.html）

- 新增 `tsinghua` 颜色系列（50 ~ 900）
- 新增动画：fade-in, scale-in, slide-up, flow-pulse, node-glow
- 新增 Google Fonts：Inter, Noto Sans SC

5.4 认证方式

字段	值
管理员账号	`admin@mails.tsinghua.edu.cn`
管理员密码	`BIDBIDBID88`
认证方式	本地硬编码（authService.ts），无 Supabase

🚀 六、部署流程（QUICK_DEPLOY.sh）

🔑 配置 SSH 密钥 → 🔌 测试服务器连接 → 📤 rsync 上传代码 → 🔧 远程 deploy.sh → 🎉 上线完成

步骤	命令/行为	说明
1. 密钥配置	`cp server_ssh_key ~/.ssh/business_server`	将项目目录下的私钥复制到 SSH 标准位置
2. 连接测试	`ssh root@8.145.43.200`	验证网络和密钥可达性
3. 代码上传	`rsync -avz`	增量同步，排除 node_modules / .git / data.db / 密钥
4. 远程部署	`deploy.sh`	在服务器 ~/workspace/business 执行构建 + PM2 重启

服务器信息：

- IP：`8.145.43.200`
- 用户：`root`
- 部署目录：`~/workspace/business`
- 访问地址：`http://aiforbazi.cloud`
- 日志查看：`pm2 logs bizmodel-backend`

rsync 排除列表（不会上传的文件）

- `node_modules/` — 服务器端 npm install 重新安装
- `dist/` — 服务器端重新 build
- `.git/` — 版本控制目录
- `backend/data.db` — 生产数据库不覆盖
- `server_ssh_key*` — 部署密钥不上传

☑ 七、部署前检查清单

✔	检查项	状态 / 说明
✔	screener/ 子目录全部文件就位	已完成 20 组件 + 14 服务 + 2 数据文件
✔	后端 screenerRoutes.js 创建并挂载	已完成

✔	Supabase 依赖完全移除	已完成 package.json + importmap
✔	12 项数据全部后端同步	已完成
✔	TypeScript 编译无新增错误	已完成
✔	导航栏「招生」入口可点击跳转	已验证
✔	管理员登录功能正常	已验证
✔	页面滚动正常	已修复

⚠ 八、注意事项

⚠️ **.env.local 的 VITE_BACKEND_URL**
当前设为 `http://localhost:3001` 。如果生产环境前后端同机部署，无需修改；如果分离部署，需改为实际后端地址。

🔒 **管理员密码为硬编码**
`authService.ts` 中管理员账密直接写在代码里。如需更改，需修改源码并重新部署。建议后续迁移至数据库或环境变量。

🗃️ **生产数据库不会被覆盖**
`upload-to-server.sh` 排除了 `backend/data.db` ，所以每次部署不会覆盖生产数据。首次部署时数据库会自动创建空表。

🔌 **部署前提**
运行 `QUICK_DEPLOY.sh` 前需要确认：(1) 项目根目录存在 `server_ssh_key` 文件；(2) 本机已安装 rsync 和 ssh 客户端。

🗑️ 九、已移除的依赖

依赖	原用途	替代方案
`@supabase/supabase-js`	Supabase 数据库 + 认证客户端	apiFetch() → Express API + 本地认证

同时从 `index.html` 的 importmap 中移除了对应的 CDN 引用，从 `.env.local` 中移除了 `VITE_SUPABASE_URL` 和 `VITE_SUPABASE_ANON_KEY`。